App Development Project Report

App Dev Project Report

1. Student Details

Name: JANNU
Roll Number: 23F2003556
Email: 23f2003556@ds.study.iitm.ac.in
About Me: I'm a student at IIT Madras, pursuing the 4 year Bachelor of Science(BS) in Data Science, and I'm currently in the diploma level of the programme.

---

2. Project Details

Project Title: Hospital Management System

Problem Statement:

Many small hospitals and clinics still rely on manual paper records, which leads to errors, lost data, and long waiting times for patients. The goal of this project was to solve this by creating a simple, computerized system. It allows the hospital to manage doctors and patients efficiently and lets patients book appointments online, reducing confusion and paperwork.

Approach:

I built this web application using Python and Flask to understand how backend web development works.

- **Modular Design:** I structured the code into different modules (Admin, Doctor, Patient) to keep the project organized and easy to maintain.
- **Database Management:** I used SQLite with SQLAlchemy to design the database. This helped me learn how to create relationships between tables, such as linking a Patient to an Appointment and a Doctor.
- **Role-Based Access:** I implemented a login system where Admins, Doctors, and Patients have different permissions. This demonstrates how to handle user authentication and secure sessions.
- **User Interface:** The frontend uses HTML, CSS, and Bootstrap to ensure the dashboards are clean and user-friendly.

---

3. AI/LLM Declaration

I used AI tools (ChatGPT/Gemini) to assist with generating initial HTML/Bootstrap boilerplate code and for debugging specific SQLAlchemy syntax errors. The extent of AI usage is limited to frontend scaffolding and resolving error messages, while the core logic and database design were implemented manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper (ORM) for managing the SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design for dashboards
Flask-Login	User authentication and session management
Flask-WTF	Secure form handling and CSRF protection
WTForms	Frontend form validation and data processing
SQLite	Lightweight local database for storing patient and doctor records
Werkzeug	Security utilities for password hashing and verification
Email-Validator	Used by WTForms to validate email address formats

---

5. Database Schema / ER Diagram

Tables:

- **User** — stores base user authentication details (id, email, password, first_name, last_name, phone)
- **Role** — stores user roles (id, name) 3)
- **Admin** — stores admin-specific profile data (id, user_id, is_super_admin)
- **Doctor** — stores doctor professional details (id, user_id, department_id, specialization, license_number)
- **Patient** — stores patient personal and medical profile (id, user_id, dob, gender, blood_group) 6)
- **Department** — stores hospital departments (id, name, description)
- **Appointment** — stores booking details (id, patient_id, doctor_id, date, time, status)
- **Treatment** — stores medical diagnosis and prescriptions (id, appointment_id, diagnosis, prescription)
- **DoctorAvailability** — stores doctor's time slots (id, doctor_id, date, start_time, end_time) 10)
- **MedicalRecord** — stores additional patient documents (id, patient_id, record_type, file_path)

Relationships:

- Many-to-Many → `User ↔ Role`
- One-to-One → `User → Admin / Doctor / Patient`
- One-to-One → `Appointment → Treatment`
- One-to-Many → `Department → Doctor`
- One-to-Many → `Doctor → Appointment`
- One-to-Many → `Patient → Appointment`
- One-to-Many → `Doctor → DoctorAvailability`
- One-to-Many → `Patient → MedicalRecord`

---

6. API Resource Endpoints

Endpoint	Method	Description
`/api/doctors`	GET	Fetch list of doctors (supports search by 'q')
`/api/doctors/<id>`	GET	Fetch specific doctor details

`/api/doctors`	POST	Create a new doctor profile (Admin only)
`/api/doctors/<id>`	PUT	Update doctor profile (Admin only)
`/api/patients`	GET	Fetch list of patients (Admin/Doctor only)
`/api/patients/<id>`	GET	Fetch specific patient details (Admin/Doctor only)
`/api/patients/<id>/history`	GET	Fetch patient medical history (Admin/Doctor only)
`/api/appointments`	GET	Fetch appointments (supports filtering)
`/api/appointments`	POST	Book a new appointment (Patient only)
`/api/appointments/<id>`	PUT	Update/Reschedule appointment (Patient only)
`/api/appointments/<id>`	DELETE	Cancel appointment (Patient only)
`/api/admin/stats`	GET	Return dashboard statistics (Admin only)
`/api/doctor/appointments_distribution`	GET	Return appointment status distribution (Doctor only)
`/api/patient/treatments`	GET	Fetch treatment history (Patient only)

---

7. Architecture and Features

Architecture:

- **app.py**: Main Flask application entry point, containing application factory, configuration loading, and all core route definitions (Web UI).
- **api.py**: Flask Blueprint defining RESTful API endpoints for external access or AJAX calls.
- **models.py**: Database models using SQLAlchemy ORM, defining the schema for Users, Roles, Doctors, Patients, Appointments, and Medical Records.
- **forms.py**: Form definitions using Flask-WTF for secure user input handling (Login, Registration, Appointments).

- **templates/**: Jinja2 HTML templates organized by role (`admin/`, `doctor/`, `patient/`) and shared layouts (`base.html`).
- **static/**: Static assets including CSS (Bootstrap), JavaScript, and images.
- **db_init.py** : Database initialization script for creating tables and seeding default roles/admin user.

Implemented Features:

Core System

- **Role-Based Access Control (RBAC):** Secure access separation for Admin, Doctor, and Patient roles.
- **User Authentication:** Secure registration and login with password hashing (Werkzeug) and session management (Flask-Login).
- **Responsive UI:** Modern, responsive interface built with Bootstrap 5.

Patient Features

- **Appointment Booking:** Patients can search for doctors and book appointments based on real-time availability.
- **Dashboard:** View upcoming and past appointments, and access medical history.
- **Medical History:** View treatments, diagnoses, and prescriptions provided by doctors.

Doctor Features

- **Doctor Dashboard:** View daily and weekly appointment schedules.
- **Availability Management:** Set and manage weekly availability slots.
- **Patient Management:** View assigned patients, update medical records, and prescribe treatments. **Appointment Handling:** Update appointment status (Complete/Cancel) and add consultation notes.

Admin Features

- **System Overview:** Dashboard with key statistics (Total Doctors, Patients, Appointments).
- **User Management:** Add, edit, or remove Doctors and Patients.
- **Data Export:** Export appointment data to CSV for external analysis.
- **Department Management:** Organize doctors into specific medical departments.

Additional Features:

- **REST API:** Comprehensive API for accessing system resources programmatically.
- **Data Validation:** Robust server-side validation for all forms and API inputs.
- **Conflict Detection:** Automated checks to prevent double-booking of doctor slots.

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1Y3NrdvldEDYtOK11LcVVI73wDhq4wom-/view?usp=sharing

---